

Phase 3 API Reference

Type System Extended API

Struct Types

`bci_type_struct_create`

```
BciTypeExt* bci_type_struct_create(const char* name);
```

Creates a new struct type with the given name.

Parameters:

- `name` : Name of the struct (copied internally)

Returns: Pointer to new struct type, or `nullptr` on allocation failure

Example:

```
BciTypeExt* point = bci_type_struct_create("Point");
```

`bci_type_struct_add_field`

```
void bci_type_struct_add_field(BciTypeExt* struct_type,  
                               const char* field_name,  
                               BciTypeExt* field_type);
```

Adds a field to a struct type.

Parameters:

- `struct_type` : Struct to add field to
- `field_name` : Name of the field
- `field_type` : Type of the field

Example:

```
bci_type_struct_add_field(point, "x", int_type);  
bci_type_struct_add_field(point, "y", int_type);
```

`bci_type_struct_finalize`

```
void bci_type_struct_finalize(BciTypeExt* struct_type);
```

Finalizes struct layout, computing final size and alignment.

Parameters:

- `struct_type` : Struct to finalize

Example:

```
bci_type_struct_finalize(point);
```

`bci_type_struct_get_field`

```
BciStructField* bci_type_struct_get_field(BciTypeExt* struct_type,
                                           const char* field_name);
```

Retrieves a field by name.

Parameters:

- `struct_type` : Struct to search
- `field_name` : Name of field to find

Returns: Pointer to field, or `nullptr` if not found

Union Types

`bci_type_union_create`

```
BciTypeExt* bci_type_union_create(const char* name, bool is_discriminated);
```

Creates a new union type.

Parameters:

- `name` : Name of the union
- `is_discriminated` : Whether union has a discriminant tag

Returns: Pointer to new union type

`bci_type_union_add_variant`

```
void bci_type_union_add_variant(BciTypeExt* union_type,
                                const char* variant_name,
                                BciTypeExt* variant_type);
```

Adds a variant to a union.

Parameters:

- `union_type` : Union to add variant to
- `variant_name` : Name of the variant
- `variant_type` : Type of the variant

Enum Types

`bci_type_enum_create`

```
BciTypeExt* bci_type_enum_create(const char* name, BciTypeExt* backing_type);
```

Creates a new enum type.

Parameters:

- `name` : Name of the enum
- `backing_type` : Integer type backing the enum

Returns: Pointer to new enum type

`bci_type_enum_add_variant`

```
void bci_type_enum_add_variant(BciTypeExt* enum_type,
                              const char* variant_name,
                              int64_t value);
```

Adds a variant to an enum.

Parameters:

- `enum_type` : Enum to add variant to
- `variant_name` : Name of the variant
- `value` : Integer value of the variant

`bci_type_enum_get_value`

```
int64_t bci_type_enum_get_value(BciTypeExt* enum_type,
                                const char* variant_name);
```

Gets the value of an enum variant.

Parameters:

- `enum_type` : Enum to query
- `variant_name` : Name of variant

Returns: Value of variant, or -1 if not found

Function Types

`bci_type_function_create`

```
BciTypeExt* bci_type_function_create(BciTypeExt* return_type, bool is_variadic);
```

Creates a new function type.

Parameters:

- `return_type` : Return type of function
- `is_variadic` : Whether function accepts variable arguments

Returns: Pointer to new function type

`bci_type_function_add_param`

```
void bci_type_function_add_param(BciTypeExt* func_type, BciTypeExt* param_type);
```

Adds a parameter to a function type.

Parameters:

- `func_type` : Function type to modify
- `param_type` : Type of parameter to add

bci_type_function_matches

```
bool bci_type_function_matches(BciTypeExt* func_type,
                              BciVec(BciTypeExt*) arg_types);
```

Checks if argument types match function signature.

Parameters:

- `func_type` : Function type to check
- `arg_types` : Vector of argument types

Returns: `true` if types match, `false` otherwise

Generic Types**bci_type_generic_create**

```
BciTypeExt* bci_type_generic_create(const char* name, BciTypeExt* base_type);
```

Creates a new generic type.

Parameters:

- `name` : Name of generic type
- `base_type` : Base type to parameterize

Returns: Pointer to new generic type

bci_type_generic_add_param

```
void bci_type_generic_add_param(BciTypeExt* generic_type, const char* param_name);
```

Adds a type parameter to a generic type.

Parameters:

- `generic_type` : Generic type to modify
- `param_name` : Name of type parameter

bci_type_generic_instantiate

```
BciTypeExt* bci_type_generic_instantiate(BciTypeExt* generic_type,
                                          BciVec(BciTypeExt*) type_args);
```

Instantiates a generic type with concrete type arguments.

Parameters:

- `generic_type` : Generic type to instantiate
- `type_args` : Vector of type arguments

Returns: Pointer to instantiated type

Type Checking

bci_type_ext_equals

```
bool bci_type_ext_equals(BciTypeExt* a, BciTypeExt* b);
```

Checks if two types are equal.

Parameters:

- `a`, `b` : Types to compare

Returns: `true` if equal, `false` otherwise

bci_type_ext_is_assignable

```
bool bci_type_ext_is_assignable(BciTypeExt* dest, BciTypeExt* src);
```

Checks if source type can be assigned to destination type.

Parameters:

- `dest` : Destination type
- `src` : Source type

Returns: `true` if assignable, `false` otherwise

Parser Extended API

Parser Initialization

parser_extended_init

```
void parser_extended_init(ParserExtended* parser, Lexer* lexer);
```

Initializes an extended parser.

Parameters:

- `parser` : Parser to initialize
- `lexer` : Lexer to use for tokens

parser_extended_parse

```
AstNode* parser_extended_parse(ParserExtended* parser);
```

Parses input and returns AST.

Parameters:

- `parser` : Parser to use

Returns: Root AST node, or `nullptr` on error

Error Handling

parser_error_at

```
void parser_error_at(ParserExtended* parser, Token* token, const char* message);
```

Reports a parse error.

Parameters:

- parser : Parser context
- token : Token where error occurred
- message : Error message

parser_synchronize

```
void parser_synchronize(ParserExtended* parser);
```

Synchronizes parser after error.

Parameters:

- parser : Parser to synchronize

Pattern Matching

ast_new_match_expr

```
AstNode* ast_new_match_expr(AstNode* scrutinee);
```

Creates a match expression.

Parameters:

- scrutinee : Expression to match against

Returns: Match expression node

ast_new_pattern_wildcard

```
AstPattern* ast_new_pattern_wildcard(void);
```

Creates a wildcard pattern.

Returns: Wildcard pattern

ast_new_pattern_binding

```
AstPattern* ast_new_pattern_binding(const char* name);
```

Creates a binding pattern.

Parameters:

- name : Variable name to bind

Returns: Binding pattern

Lambda Expressions

ast_new_lambda

```
AstNode* ast_new_lambda(void);
```

Creates a lambda expression.

Returns: Lambda expression node

ast_lambda_add_param

```
void ast_lambda_add_param(AstNode* lambda, const char* name, BciTypeExt* type);
```

Adds a parameter to a lambda.

Parameters:

- `lambda` : Lambda to modify
- `name` : Parameter name
- `type` : Parameter type (can be `nullptr` for inference)

ast_lambda_add_capture

```
void ast_lambda_add_capture(AstNode* lambda, const char* name);
```

Adds a captured variable to a lambda.

Parameters:

- `lambda` : Lambda to modify
- `name` : Variable name to capture

Semantic Analysis API

Type Inference

type_inference_init

```
void type_inference_init(TypeInferenceContext* ctx);
```

Initializes type inference context.

Parameters:

- `ctx` : Context to initialize

type_inference_new_var

```
TypeVariable* type_inference_new_var(TypeInferenceContext* ctx, const char* name);
```

Creates a new type variable.

Parameters:

- `ctx` : Inference context
- `name` : Variable name (optional)

Returns: Pointer to new type variable

type_inference_solve

```
bool type_inference_solve(TypeInferenceContext* ctx);
```

Solves type constraints.

Parameters:

- `ctx` : Inference context

Returns: `true` if successful, `false` if unsolvable

Lifetime Analysis**lifetime_analyzer_init**

```
void lifetime_analyzer_init(LifetimeAnalyzer* analyzer);
```

Initializes lifetime analyzer.

Parameters:

- `analyzer` : Analyzer to initialize

lifetime_analyze_program

```
void lifetime_analyze_program(LifetimeAnalyzer* analyzer, AstNode* program);
```

Analyzes lifetimes in a program.

Parameters:

- `analyzer` : Analyzer to use
- `program` : Program AST to analyze

lifetime_is_live

```
bool lifetime_is_live(LifetimeAnalyzer* analyzer, const char* var_name, int line);
```

Checks if a variable is live at a given line.

Parameters:

- `analyzer` : Analyzer to query
- `var_name` : Variable name
- `line` : Line number

Returns: `true` if live, `false` otherwise

Escape Analysis

escape_analyzer_init

```
void escape_analyzer_init(EscapeAnalyzer* analyzer);
```

Initializes escape analyzer.

Parameters:

- analyzer : Analyzer to initialize

escape_analyze_function

```
void escape_analyze_function(EscapeAnalyzer* analyzer, AstNode* func);
```

Analyzes escapes in a function.

Parameters:

- analyzer : Analyzer to use
- func : Function AST to analyze

escape_can_stack_allocate

```
bool escape_can_stack_allocate(EscapeAnalyzer* analyzer, const char* var_name);
```

Checks if a variable can be stack-allocated.

Parameters:

- analyzer : Analyzer to query
- var_name : Variable name

Returns: true if stack allocation is safe

Control Flow Graph

cfg_init

```
void cfg_init(ControlFlowGraph* cfg);
```

Initializes a control flow graph.

Parameters:

- cfg : CFG to initialize

cfg_build_from_ast

```
void cfg_build_from_ast(ControlFlowGraph* cfg, AstNode* program);
```

Builds CFG from AST.

Parameters:

- cfg : CFG to build
- program : Program AST

cfg_is_reachable

```
bool cfg_is_reachable(CfgNode* from, CfgNode* to);
```

Checks if one node is reachable from another.

Parameters:

- `from` : Starting node
- `to` : Target node

Returns: `true` if reachable, `false` otherwise